

The database

SPRFST carries a small relational database called Ember. There is no server to install and no driver to add: `use std.db` and you have tables, SQL and transactions.

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")

    db.exec(store, "create table people (id int, name text, city text, score num)")
    db.exec(store, "insert into people values (1, 'Ada', 'London', 99.5)")
    db.exec(store, "insert into people values (2, 'Grace', 'New York', 97.0)")
    db.exec(store, "insert into people values (3, 'Alan', 'London', 95.5)")

    for row in db.query(store, "select name, city from people order by name") {
        io.say("{row.get(\"name\") ?? \"?\"} - {row.get(\"city\") ?? \"?\"}")
    }

    db.close(store)
}
```

`:memory:` keeps everything in memory. Give a file path instead and Ember writes through on every change, so the data is there next time.

Querying

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")
    db.exec(store, "create table fruit (name text, crates int)")
    db.exec(store, "insert into fruit values ('apples', 12), ('pears', 4), ('plums', 0)")

    let in_stock = db.query(store, "select name from fruit where crates > 0 order by crates desc")
    io.say("{in_stock}")

    let counted = db.query(store, "select count(*) from fruit")
    io.say("{counted.first()?.get(\"count\") ?? 0} kinds")

    let like = db.query(store, "select name from fruit where name like 'p%'")
    io.say("{like}")

    db.close(store)
}
```

Ember understands `create table`, `drop table`, `insert`, `select` with `where`, `order by` and `limit`, `update` and `delete`. Conditions support `=` `!=` `<` `>` `<=` `>=` and `like`, joined with `and` and `or`

Transactions

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")
    db.exec(store, "create table t (n int)")
    db.exec(store, "insert into t values (1), (2), (3)")

    db.begin(store)
    db.exec(store, "delete from t")
    io.say("inside: {db.query(store, "select count(*) from t").first()?.get("count") ?
? 0}")
    db.rollback(store)
    io.say("after rollback: {db.query(store, "select count(*) from t").first()?.get("c
ount") ?? 0}")

    db.close(store)
}
```

`db.begin` takes a snapshot. `db.commit` keeps the changes and writes them out; `db.rollback` puts the snapshot back.

Exercise

Build a tiny address book: a table, a function to add a person, a function to search by the start of a name, and a count of the total.